



ContaQuiz

Documento de Requisitos e Especificação Técnica (PRD)
Refatoração Completa da Camada de Backend

Equipe de Desenvolvimento
Techtins

Conteúdo

1	Introdução	1
1.1	Sobre o ContaQuiz	1
1.2	Público-Alvo e Atores do Sistema	1
1.3	Contexto do Projeto de Refatoração	1
2	Motivação para a Refatoração	1
2.1	Limitações da Stack Legada (AS-IS)	2
2.2	Objetivos e Benefícios do Modelo Alvo (TO-BE)	2
3	Stack Tecnológica	3
4	Arquitetura do Novo Backend	3
4.1	Visão Geral das Camadas	3
4.2	Padrões de Projeto e Diretrizes de Desenvolvimento	4
4.2.1	Isolamento de Payload com DTOs	4
4.2.2	Idioma de Código	4
4.3	Tratamento Global de Erros e Exceções	4
4.3.1	Estrutura Padrão de Erro (ApiError)	4
5	Modelo de Dados	5
5.1	Entidades do Domínio	5
5.2	Relacionamentos Chave	6
5.3	Enums de Negócio	6
6	Módulos Funcionais e Contratos de API	6
6.1	Responsabilidades dos Módulos	7
6.2	Tabela Geral de Endpoints da API	7
7	Documentos de Referência	8

1 Introdução

1.1 Sobre o ContaQuiz

O **ContaQuiz** é uma plataforma web voltada para a preparação de candidatos a exames de certificação e concursos públicos na área contábil, com especial destaque para o Exame de Suficiência do Conselho Federal de Contabilidade (CFC/CRC). A ferramenta foi concebida para fornecer um ambiente prático e dinâmico de estudos, onde os usuários podem simular exames reais, testar seus conhecimentos em disciplinas específicas e monitorar sua evolução acadêmica.

1.2 Público-Alvo e Atores do Sistema

O ecossistema do ContaQuiz é estruturado em torno de dois perfis principais de usuários:

- **Alunos (Candidatos):** Usuários finais que utilizam a plataforma para resolver simulados (quizzes) gerados manualmente ou dinamicamente, favoritar questões complexas para revisão posterior e analisar métricas detalhadas de desempenho por disciplina e tópico.
- **Administradores (Equipe Pedagógica/Gestão):** Responsáveis pela curadoria do conteúdo, incluindo o cadastro e manutenção de disciplinas, temas, subtemas hierárquicos e questões de múltipla escolha ou certo/errado, além do monitoramento de estatísticas gerais de uso da plataforma.

1.3 Contexto do Projeto de Refatoração

A presente documentação formaliza as especificações para a **refatoração completa da camada de backend** do ContaQuiz. O sistema original, construído sobre uma arquitetura Node.js com Express e persistência em MongoDB (NoSQL), enfrentava desafios de consistência de dados, ausência de tipagem forte e dificuldades de manutenção a longo prazo devido à falta de padronização estrutural.

A nova arquitetura (visão *TO-BE*) migra o núcleo do sistema para o ecossistema **Quarkus (Java 21)** integrado ao banco de dados relacional **PostgreSQL 16**. O principal direcionador deste projeto é o **Princípio de Substituição Transparente**: a camada de *frontend* (React/Next.js) deve permanecer inalterada, exigindo que o novo *backend* replique com exatidão todos os contratos de API, comportamentos de negócios e fluxos de dados estabelecidos no sistema legado.

2 Motivação para a Refatoração

A decisão de refatorar o *backend* do ContaQuiz é fundamentada na necessidade de sanar limitações técnicas acumuladas no ciclo de vida do software legado. Esta seção detalha os principais gargalos identificados e as melhorias esperadas com a transição tecnológica.

2.1 Limitações da Stack Legada (AS-IS)

A arquitetura anterior, baseada em Node.js com Express e MongoDB, apresentava entraves críticos para o crescimento da plataforma:

- **Inconsistência de Dados e Falta de Esquema Rígido:** Por ser um banco de dados NoSQL, o MongoDB permitiu que documentos fossem inseridos com estruturas heterogêneas ao longo do tempo. A ausência de integridade referencial a nível de banco gerou registros órfãos (como questões associadas a disciplinas que foram removidas ou históricos de tentativas com referências corrompidas).
- **Baixa Padronização de Projeto:** O Express é um *framework* minimalista que não impõe uma estrutura de pastas ou um padrão arquitetural rígido. Sem diretrizes estritas, a base de código tornou-se acoplada, dificultando a implementação de testes automatizados e o *onboarding* de novos desenvolvedores.
- **Ausência de Tipagem Forte:** O uso de JavaScript puro (sem TypeScript) elevou a incidência de erros em tempo de execução (*runtime*), comumente causados por propriedades ausentes nos payloads ou incompatibilidade de tipos nas operações de negócio.
- **Evolução Não Documentada (*Shadow Features*):** A falta de uma modelagem formal e de sincronia entre os requisitos originais e a codificação fez com que funcionalidades inteiras (como regras dinâmicas de geração de simulados e o motor de *analytics* de desempenho) fossem criadas sem documentação de apoio, operando como caixas-pretas de difícil manutenção.

2.2 Objetivos e Benefícios do Modelo Alvo (TO-BE)

A migração para a stack Java 21 com Quarkus e PostgreSQL visa atingir os seguintes benefícios de engenharia:

- **Integridade e Consistência com Banco Relacional:** O PostgreSQL garantirá a consistência dos dados do ContaQuiz por meio de restrições de chaves primárias, chaves estrangeiras (*Foreign Keys*), índices de unicidade e transações ACID. Isso previne nativamente a corrupção de históricos e o surgimento de registros inconsistentes.
- **Segurança de Tipos e Robustez:** A compilação estrita do Java 21, combinada com a tipagem forte do ecossistema Quarkus, permite identificar possíveis erros de lógica, mapeamento e integração antes mesmo de o código ser implantado em produção.
- **Arquitetura Orientada a Contratos e Camadas:** O novo *backend* adota padrões de projeto claros e bem definidos (Resource, Service, Repository, Model). O isolamento de responsabilidades facilita a manutenção evolutiva do software, a escrita de testes de integração robustos e a auditoria de regras de negócio.
- **Eficiência Operacional em Containers:** O Quarkus foi projetado sob o conceito de *GraalVM/Native Compilation* e *Build-time Optimization*. Ele oferece um tempo de inicialização (*startup time*) extremamente baixo e consumo reduzido de memória RAM se comparado a servidores Java tradicionais, otimizando o uso de recursos em ambientes de computação em nuvem e containers (Docker).

3 Stack Tecnológica

A transição da infraestrutura de software do ContaQuiz foi planejada para substituir tecnologias de alta flexibilidade por ferramentas de alta confiabilidade e padronização corporativa. A Tabela 1 resume as mudanças realizadas na camada de *backend*.

Tabela 1: Comparativo de Tecnologias (AS-IS × TO-BE)

Componente	Stack Legada (AS-IS)	Stack Alvo (TO-BE)
Linguagem e Runtime	JavaScript (Node.js)	Java 21 (Quarkus/JVM)
Framework Backend	Express	Quarkus (REST Jackson)
Banco de Dados	MongoDB (NoSQL Document)	PostgreSQL 16 (Relacional)
Acesso a Dados / ORM	Mongoose	Hibernate ORM com Panache
Segurança / JWT	Middleware Express Customizado	SmallRye JWT
Segurança de Senhas	BCrypt (JS)	Elytron Security Properties File
Documentação de API	Indisponível (Manual)	SmallRye OpenAPI (Swagger UI)
Orquestração Local	Execução Direta (Local)	Docker & Docker Compose

4 Arquitetura do Novo Backend

O novo *backend* do ContaQuiz é estruturado sob o paradigma de camadas bem definidas, eliminando acoplamentos desnecessários e garantindo a separação de responsabilidades. O projeto adota uma abordagem direta **sem interfaces intermediárias para serviços**, simplificando a base de código sem comprometer a testabilidade.

4.1 Visão Geral das Camadas

O fluxo de uma requisição HTTP e a respectiva resposta são processados através de quatro camadas lógicas concretas:

1. **Resource (Controller):** Porta de entrada das requisições HTTP (JAX-RS). Suas responsabilidades são estritamente limitadas a:
 - Receber a requisição e mapear os parâmetros de rota, *query string* ou corpo.
 - Validar sintaticamente o DTO (*Data Transfer Object*) de entrada usando validações declarativas (*Hibernate Validator*).
 - Converter o DTO de entrada em uma Entidade de Domínio pura.
 - Invocar o método correspondente na camada de *Service*.
 - Capturar o retorno da camada de negócio (Entidade), convertê-lo no DTO de resposta correspondente e retornar o status HTTP adequado.
2. **Service:** Concentra as regras de negócio e o controle transacional do sistema. Suas responsabilidades incluem:
 - Executar validações lógicas e regras de negócio do ContaQuiz.
 - Operar **exclusivamente com entidades de domínio puras** e tipos primitivos (não processa ou retorna DTOs).

- Coordenar transações de banco de dados e integrar múltiplos repositórios se necessário.
3. **Repository:** Camada encarregada das operações de acesso a dados, implementando a interface *PanacheRepository*<*T*>. Suas responsabilidades são:
 - Encapsular consultas SQL/HQL customizadas, filtros complexos e paginações.
 - Isolar a lógica de banco de dados das regras de negócio do *Service* e do mapeamento do *Model*.
 4. **Model (Entities):** Representação orientada a objetos das tabelas relacionais do banco PostgreSQL. Todas as entidades herdam obrigatoriamente da classe abstrata *BaseEntity*, que padroniza os campos de auditoria (*createdAt*, *updatedAt*) e controle de status ativo (*active*).

4.2 Padrões de Projeto e Diretrizes de Desenvolvimento

4.2.1 Isolamento de Payload com DTOs

Para impedir o vazamento de estruturas internas do banco de dados para a rede, é obrigatório o uso de DTOs nas requisições e respostas. Os DTOs são subdivididos em pacotes específicos de direção:

- *dto.request*: Objetos imutáveis que mapeiam dados vindos do *frontend* (ex: *LoginRequestDTO*).
- *dto.response*: Objetos projetados para retorno ao cliente (ex: *UserResponseDTO*), garantindo que apenas dados necessários sejam transmitidos.

4.2.2 Idioma de Código

Todo o código-fonte (nomes de classes, atributos, métodos, tabelas SQL, colunas e logs de console) deve ser escrito em **inglês**. As únicas exceções são as mensagens de erro retornadas nas telas do *frontend* e o conteúdo das questões contábeis cadastradas, que devem ser mantidos em português (PT-BR).

4.3 Tratamento Global de Erros e Exceções

Para evitar a exposição de detalhes de infraestrutura (como pilhas de erro ou *stack-traces* do Java), o sistema intercepta exceções por meio de tratadores globais do JAX-RS (*ExceptionHandler*).

4.3.1 Estrutura Padrão de Erro (ApiError)

Toda falha processada pelo sistema é serializada em um formato JSON padronizado com os seguintes campos:

- **timestamp:** Data e hora exatas da ocorrência da falha.
- **status:** Código de status HTTP representativo do erro.
- **error:** Descrição textual curta da categoria do erro (ex: “Not Found”).

- **message:** Mensagem explicativa e amigável destinada ao usuário final (PT-BR).
- **path:** O *endpoint* URI que originou o erro.

5 Modelo de Dados

O novo modelo de persistência baseia-se em uma modelagem relacional normalizada, projetada para assegurar integridade referencial absoluta no PostgreSQL 16. O mapeamento relacional elimina a flexibilidade inconsistente do modelo legado (*NoSQL*) e consolida os relacionamentos lógicos do domínio.

5.1 Entidades do Domínio

As tabelas de banco de dados e suas respectivas classes Java são descritas a seguir:

- **User (tb_user):** Armazena os dados cadastrais de alunos e administradores. Assegura a unicidade do e-mail no banco.
- **Discipline (tb_discipline):** Representa as grandes áreas do conhecimento contábil (ex: Contabilidade Geral, Custos, Auditoria).
- **Topic (tb_topic):** Representa os assuntos específicos vinculados a uma disciplina. Utiliza um relacionamento reflexivo (*parent_topic_id*) para suportar recursividade infinita de subtemas.
- **Question (tb_question):** Armazena o enunciado e metadados das perguntas do simulado.
- **QuestionOption (tb_question_option):** Normaliza as alternativas das questões. Cada questão possui múltiplas linhas nesta tabela, contendo a flag de indicação da alternativa correta.
- **Quiz (tb_quiz):** Guarda os cabeçalhos de simulados cadastrados ou dinâmicos, definindo tempo de prova, nota de corte e visibilidade.
- **QuizResult (tb_quiz_result):** Registra a nota, tempo de realização e estatísticas consolidadas da submissão de um aluno para um simulado específico.
- **QuizResultCorrection (tb_quiz_result_correction):** Detalha a resposta dada pelo aluno para cada questão individual do simulado no momento da submissão, incluindo uma cópia (*snapshot*) da explicação pedagógica para garantir rastreabilidade histórica.
- **QuestionFavorite (tb_question_favorite):** Tabela associativa que viabiliza a funcionalidade de favoritar questões, com restrição de unicidade para impedir favoritos duplicados por usuário para a mesma questão.

5.2 Relacionamentos Chave

A coesão dos dados baseia-se nos seguintes vínculos estruturais:

- **1 para N (1:N):**

- Uma Disciplina possui muitos Tópicos (*tb_discipline* → *tb_topic*).
- Uma Questão contém muitas Alternativas (*tb_question* → *tb_question_option*).
- Um Resultado de Quiz agrupa muitas Correções Detalhadas (*tb_quiz_result* → *tb_quiz_result_correction*).

- **N para M (N:M):**

- Uma Questão pertence a múltiplos Tópicos/Subtemas (mapeada pela tabela associativa intermediária *tb_question_topic*).
- Um Quiz é composto por várias Questões, e uma Questão pode pertencer a vários Quizzes (mapeada pela tabela intermediária *tb_quiz_question*).

5.3 Enums de Negócio

As propriedades de comportamento das entidades são limitadas por domínios fechados (Enums), persistidos como strings (*EnumType.STRING*) para legibilidade direta das tabelas no banco de dados:

- **UserSystemRole:** Define os privilégios de acesso do usuário logado.
 - *ADMIN*: Acesso total às rotas de gerenciamento acadêmico e visualização de logs.
 - *ALUNO*: Restrito à resolução de provas e consulta de relatórios próprios.
- **QuestionType:** Especifica a sistemática de preenchimento e resposta.
 - *MULTIPLA_ESCOLHA*: Exige o envio do índice da alternativa selecionada.
 - *CERTO_ERRADO*: Limita a seleção à avaliação binária de uma afirmação.
- **DifficultyLevel:** Classifica o grau de complexidade para indexação e busca.
 - Valores aceitos: *FACIL*, *MEDIO*, *DIFICIL*.
- **QuizVisibility:** Controla o acesso de visualização e realização de um simulado.
 - *PUBLIC*: Visível globalmente para qualquer usuário logado ou visitante.
 - *PRIVATE*: Restrito à visualização interna do aluno proprietário do simulado dinâmico.

6 Módulos Funcionais e Contratos de API

O sistema do ContaQuiz é dividido de forma modular para agrupar as responsabilidades de negócio. Os cinco módulos principais gerenciam desde a segurança do acesso até a inteligência analítica de desempenho dos alunos.

6.1 Responsabilidades dos Módulos

1. **Módulo de Autenticação (Auth):** Controla o ciclo de vida da sessão do usuário. Gerencia o registro de novos alunos, a validação de credenciais de login e a emissão de tokens JWT, além de prover o endpoint seguro de identificação do perfil logado (*/me*).
2. **Módulo Administrativo de Conteúdo (CRUDs):** Centraliza a gestão dos dados acadêmicos estruturais. Restringe a criação, atualização e exclusão física ou lógica de Disciplinas, Tópicos/Subtemas e Questões aos usuários com papel administrativo (*ADMIN*).
3. **Módulo de Quizzes (Simulados):** Viabiliza a confecção de exames. Suporta a criação estática de simulados (via administrador) e o motor de geração automática de simulados sob demanda, que sorteia questões randômicas do banco usando filtros de disciplina, tópicos e dificuldade informados pelo aluno.
4. **Módulo de Resolução e Histórico (Analytics):** Corrige automaticamente os quizzes submetidos pelos alunos comparando os mapeamentos de respostas com o gabarito. Consolida estatísticas de acertos por disciplina, tópicos e níveis de dificuldade, identificando pontos críticos de estudo (áreas fracas) do aluno.
5. **Módulo de Favoritos e Estatísticas Auxiliares:** Permite ao aluno marcar questões complexas para posterior revisão pedagógica de forma ágil, e oferece endpoints informativos com o volume de dados geral da plataforma para o painel de controle administrativo.

6.2 Tabela Geral de Endpoints da API

A Tabela 2 consolida os contratos de rotas definidos para o *backend* (TO-BE), mantendo a perfeita compatibilidade com as chamadas de rede do *frontend* legado.

Tabela 2: Contratos de Endpoints da API ContaQuiz

Módulo	Método	Rota	Autenticação
Auth	POST	/api/auth/login	Público
Auth	POST	/api/auth/register	Público
Auth	GET	/api/auth/me	JWT (Qualquer)
Disciplinas	GET	/api/disciplinas	JWT (Qualquer)
Disciplinas	GET	/api/disciplinas/:id	JWT (Qualquer)
Disciplinas	POST	/api/disciplinas	JWT (ADMIN)
Disciplinas	PUT	/api/disciplinas/:id	JWT (ADMIN)
Disciplinas	DELETE	/api/disciplinas/:id	JWT (ADMIN)
Temas	GET	/api/temas	JWT (Qualquer)
Temas	POST	/api/temas	JWT (ADMIN)
Questões	GET	/api/questoes	JWT (Qualquer)
Questões	POST	/api/questoes	JWT (ADMIN)
Quizzes	GET	/api/quizzes	Opcional (JWT/Público)
Quizzes	GET	/api/quizzes/:id	Opcional (JWT/Público)
Quizzes	POST	/api/quizzes	JWT (ADMIN)
Quizzes	POST	/api/quizzes/generate	JWT (Qualquer)
Quizzes	PUT	/api/quizzes/:id	JWT (ADMIN)
Quizzes	DELETE	/api/quizzes/:id	JWT (ADMIN)
Resultados	POST	/api/quizzes/:id/submit	JWT (Qualquer)
Resultados	GET	/api/quiz-results/:resultId	Opcional (JWT/Público)
Histórico	GET	/api/historico	JWT (Qualquer)
Favoritos	GET	/api/question-favorites	JWT (Qualquer)
Favoritos	POST	/api/questions/:questionId/favorite	JWT (Qualquer)
Admin Stats	GET	/api/admin/stats	JWT (ADMIN)

7 Documentos de Referência

Para o aprofundamento nos detalhes de implementação técnica de cada caso de uso, banco de dados ou payload específico, os desenvolvedores devem consultar a documentação complementar contida no diretório *docs/* do repositório:

docs/api_contracts.md: Define detalhadamente a assinatura de todos os endpoints da API. Contém exemplos exatos de JSON para requisições de entrada, headers de autenticação, respostas de sucesso (*200 OK*, *201 Created*) e de erros previstos (*400 Bad Request*, *401 Unauthorized*, *404 Not Found*).

docs/arquitetura_e_setup.md: Guia prático de inicialização (*onboarding*) do ambiente local. Contém as configurações do arquivo *application.properties*, a orquestração do banco PostgreSQL no Docker Compose, instruções de uso do Makefile, padrões para criação de DTOs e código de referência do interceptador global de erros.

docs/modelagem_banco_relacional.md: Documento de transição NoSQL para SQL. Apresenta o mapeamento detalhado das tabelas físicas do PostgreSQL (colunas, chaves primárias, estrangeiras e restrições de unicidade) e seu espelhamento para classes de entidade Java JPA, acompanhado do Diagrama Entidade-Relacionamento (DER).

`docs/shadow_features.md`: Mapeamento detalhado das regras de negócio que foram codificadas na aplicação legada mas não possuíam especificação formal, como o algoritmo de distribuição proporcional de questões na geração dinâmica de quizzes e a fórmula de consolidação de desempenho do aluno.

`docs/plano_refatoracao_backend.md`: Cronograma geral do projeto. Detalha os objetivos e as ações tomadas em cada uma das fases preparatórias e o planejamento de distribuição de tarefas da equipe.

`docs/[Escopo] ContaQuiz.docx` e **User Stories** (`[US_0X]`): Documentações de escopo funcionais originais do projeto ContaQuiz. Servem como fonte de consulta histórica para o entendimento das intenções originais de negócio do produto.